

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО
«ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ КАФЕДРА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №7
по дисциплине
«Информатика и основы программирования»

Студент
гр. БИН-25-3

С.Д. Мурадуллаева

Ассистент
преподавателя

М.В. Водяницкий

Задание

Выполнить задания на Python и оформить отчет по стандартам ВВГУ.

Задание 1

Имеется список объектов Фонда с указанием уровня угрозы:

```
objects = [  
    ("Containment Cell A", 4),  
    ("Archive Vault", 1),  
    ("Bio Lab Sector", 3),  
    ("Observation Wing", 2)  
]
```

Используя sorted и лямбда-выражение, отсортируйте объекты по возрастанию уровня угрозы

Задание 2

Дан список сотрудников Фонда с количеством проведенных смен и стоимостью одной смены:

```
staff_shifts = [  
    {"name": "Dr. Shaw", "shift_cost": 120, "shifts": 15},  
    {"name": "Agent Torres", "shift_cost": 90, "shifts": 22},  
    {"name": "Researcher Hall", "shift_cost": 150, "shifts": 10}  
]
```

Используя map и лямбда-выражение, создайте список общей стоимости работы каждого сотрудника

Затем найдите максимальную стоимость с помощью max

Задание 3

Дан список персонала с уровнем допуска:

```
personnel = [  
    {"name": "Dr. Klein", "clearance": 2},  
    {"name": "Agent Brooks", "clearance": 4},  
    {"name": "Technician Reed", "clearance": 1}  
]
```

Используя map и лямбда-выражение, создайте новый список, где каждому сотруднику добавляется категория допуска:

- "Restricted" - уровень 1
- "Confidential" - уровни 2–3

- "Top Secret" - уровень 4 и выше

Результат должен быть списком словарей

Задание 4

Дан список зон Фонда с указанием времени активности (в часах):

```
zones = [  
    {"zone": "Sector-12", "active_from": 8, "active_to": 18},  
    {"zone": "Deep Storage", "active_from": 0, "active_to": 24},  
    {"zone": "Research Wing", "active_from": 9, "active_to": 17}  
]
```

Используя filter и лямбда-выражение, выберите зоны, которые полностью работают в дневной период (с 8 до 18 включительно)

Задание 5

Фонд анализирует служебные отчеты. Некоторые отчеты содержат внешние ссылки, которые должны быть удалены перед архивированием

```
reports = [  
    {"author": "Dr. Moss", "text": "Analysis completed. Reference: http://external-  
        archive.net"},  
    {"author": "Agent Lee", "text": "Incident resolved without escalation."},  
    {"author": "Dr. Patel", "text": "Supplementary data available at https://secure-  
        research.org"},  
    # ... остальные отчеты из задания  
]
```

Используя filter и лямбда-выражение:

1. Отберите отчеты, содержащие ссылки (http или https)
2. Преобразуйте их так, чтобы вместо ссылки отображалось [ДАННЫЕ

УДАЛЕНЫ]

Задание 6

Дан список SCP-объектов с указанием их класса содержания:

```
scp_objects = [  
    {"scp": "SCP-096", "class": "Euclid"},  
    {"scp": "SCP-173", "class": "Euclid"},  
    {"scp": "SCP-055", "class": "Keter"},  
    {"scp": "SCP-999", "class": "Safe"},  
    {"scp": "SCP-3001", "class": "Keter"}  
]
```

Используя filter и лямбда-выражение, сформируйте список SCP-объектов, которые требуют усиленных мер содержания

□ К объектам с усиленными мерами относятся все SCP, класс которых не равен "Safe"

Результат должен быть списком словарей исходного формата

Задание 7

Дан список инцидентов с количеством задействованного персонала:

```
incidents = [  
    {"id": 101, "staff": 4}, {"id": 102, "staff": 12},  
    {"id": 103, "staff": 7},  
    {"id": 104, "staff": 20}  
]
```

Используя sorted и лямбда-выражение:

1. Отсортируйте инциденты по количеству персонала
2. Оставьте только три наиболее ресурсоемких инцидента

Задание 8

Дан список протоколов безопасности и их уровней критичности:

```
protocols = [  
    ("Lockdown", 5),  
    ("Evacuation", 4),  
    ("Data Wipe", 3),  
    ("Routine Scan", 1)  
]
```

Используя map и лямбда-выражение, создайте новый список строк вида:

"Protocol Lockdown - Criticality 5"

Задание 9

Имеется список смен охраны с указанием длительности (в часах):

```
shifts = [6, 12, 8, 24, 10, 4]
```

Используя filter и лямбда-выражение, выберите только те смены, которые:

- длятся не менее 8 часов
- не превышают 12 часов

Задание 10

Дан список сотрудников с результатами психологической оценки (от 0 до 100):

```
evaluations = [  
    {"name": "Agent Cole", "score": 78},
```

```
{ "name": "Dr. Weiss", "score": 92 },  
{ "name": "Technician Moore", "score": 61 },  
{ "name": "Researcher Lin", "score": 88 }  
]
```

Используя `max` и лямбда-выражение, определите сотрудника с наивысшей оценкой
Результатом должно быть имя сотрудника и его балл

Содержание

1	Выполнение работы	7
1.1	Задание 1	7
1.2	Задание 2	7
1.3	Задание 3	8
1.4	Задание 4	9
1.5	Задание 5	10
1.6	Задание 6	11
1.7	Задание 7	11
1.8	Задание 8	12
1.9	Задание 9	13
1.10	Задание 10	13

1 Выполнение работы

1.1 Задание 1

Программа сортирует список объектов Фонда по возрастанию уровня угрозы, используя функцию `sorted()` с лямбда-выражением в качестве ключа сортировки.

```
1 objects = [  
2     ("Containment Cell A", 4),  
3     ("Archive Vault", 1),  
4     ("Bio Lab Sector", 3),  
5     ("Observation Wing", 2)  
6 ]  
7  
8 sorted_objects = sorted(objects, key=lambda x: x[1])  
9  
10 print("Объекты, отсортированные по уровню угрозы:")  
11 for obj in sorted_objects:  
12     print(f"{obj[0]} – Уровень угрозы: {obj[1]}")
```

Рисунок 1 – листинг программы для задания 1

1. Создан список кортежей `objects`, где каждый кортеж содержит название объекта и его уровень угрозы
2. Использована функция `sorted()` с параметром `key=lambda x: x[1]`, которая указывает, что сортировка должна выполняться по второму элементу кортежа (уровню угрозы)
3. Результат выводится в удобном требуемом формате.

1.2 Задание 2

Программа рассчитывает общую стоимость работы каждого сотрудника на основе стоимости одной смены и количества смен, а затем находит сотрудника с максимальной общей стоимостью.

```
1 staff_shifts = [  
2     {"name": "Dr. Shaw", "shift_cost": 120, "shifts": 15},  
3     {"name": "Agent Torres", "shift_cost": 90, "shifts": 22},  
4     {"name": "Researcher Hall", "shift_cost": 150, "shifts": 10}  
5 ]  
6  
7 total_costs = list(map(lambda emp: {  
8     "name": emp["name"],  
9     "total_cost": emp["shift_cost"] * emp["shifts"]  
10 }, staff_shifts))  
11  
12 max_cost_info = max(total_costs, key=lambda x: x["total_cost"])  
13  
14 print("Общая стоимость работы каждого сотрудника:")  
15 for cost in total_costs:  
16     print(f"{cost['name']}: ${cost['total_cost']}")  
17  
18 print(f"\nМаксимальная стоимость: {max_cost_info['name']} -  
    ${max_cost_info['total_cost']}")
```

Рисунок 2 – листинг программы для задания 2

- Использована функция map() с лямбда-выражением для преобразования исходных данных: для каждого сотрудника создается словарь с именем и общей стоимостью
- Применена функция max() с ключом key=lambda x: x["total_cost"] для поиска записи с максимальной стоимостью

1.3 Задание 3

Программа добавляет категорию допуска для каждого сотрудника на основе его уровня допуска, используя условные выражения внутри лямбда-функции.


```

1 personnel = [
2     {"name": "Dr. Klein", "clearance": 2},
3     {"name": "Agent Brooks", "clearance": 4},
4     {"name": "Technician Reed", "clearance": 1}
5 ]
6
7 personnel_with_category = list(map(lambda p: {
8     "name": p["name"],
9     "clearance": p["clearance"],
10    "category": (
11        "Restricted" if p["clearance"] == 1 else
12        "Confidential" if 2 <= p["clearance"] <= 3 else
13        "Top Secret"
14    )
15 }, personnel))
16
17 print("Персонал с категориями допуска:")
18 for person in personnel_with_category:
19     print(f"{person['name']}: Уровень {person['clearance']} – {person['category']}")

```

Рисунок 3 – листинг программы для задания 3

- Использована функция map() с лямбда-выражением, которое создает новый словарь для каждого сотрудника
- В лямбда-выражении применено условное выражение (тернарный оператор) для определения категории допуска:
 1. "Restricted" для уровня 1
 2. "Confidential" для уровней 2-3
 3. "Top Secret" для уровня 4 и выше

1.4 Задание 4

Программа фильтрует зоны Фонда, оставляя только те, которые полностью работают в дневной период с 8:00 до 18:00 включительно.

```

1 zones = [
2     {"zone": "Sector-12", "active_from": 8, "active_to": 18},
3     {"zone": "Deep Storage", "active_from": 0, "active_to": 24},
4     {"zone": "Research Wing", "active_from": 9, "active_to": 17}
5 ]
6
7 daytime_zones = list(filter(
8     lambda z: z["active_from"] >= 8 and z["active_to"] <= 18,
9     zones
10 ))
11
12 print("Зоны, работающие в дневной период (8:00–18:00):")
13 for zone in daytime_zones:
14     print(f"{zone['zone']}: с {zone['active_from']}:00 до {zone['active_to']}:00")

```

Рисунок 4 – листинг программы для задания 4

- Использована функция `filter()` с лямбда-выражением в качестве условия фильтрации
- Лямбда-выражение проверяет, что время начала активности не ранее 8:00 (`active_from >= 8`) и время окончания не позднее 18:00 (`active_to <= 18`)

1.5 Задание 5

Программа выполняет два этапа обработки отчетов: сначала отбирает отчеты, содержащие внешние ссылки, затем заменяет все ссылки в этих отчетах на текст "[ДАННЫЕ УДАЛЕНЫ]".

```

1 reports = [
2     {"author": "Dr. Moss", "text": "Analysis completed. Reference: http://external-archive.net"},
3     {"author": "Agent Lee", "text": "Incident resolved without escalation."},
4     {"author": "Dr. Patel", "text": "Supplementary data available at https://secure-research.org"},
5     {"author": "Supervisor Kane", "text": "No anomalies detected during inspection."},
6     {"author": "Researcher Bloom", "text": "Extended observations uploaded to http://research-notes.lab"},
7     {"author": "Agent Novak", "text": "Perimeter secured. No external interference observed."},
8     {"author": "Dr. Hargreeve", "text": "Full containment log stored at https://internal-db.scp"},
9     {"author": "Technician Moore", "text": "Routine maintenance completed successfully."},
10    {"author": "Dr. Alvarez", "text": "Cross-reference materials: http://crosslink.foundation"},
11    {"author": "Security Officer Tan", "text": "Shift completed without incidents."},
12    {"author": "Analyst Wright", "text": "Statistical model published at https://analysis-hub.org"},
13    {"author": "Dr. Kowalski", "text": "Behavioral deviations documented internally."},
14    {"author": "Agent Fischer", "text": "Additional footage archived: http://video-storage.sec"},
15    {"author": "Senior Researcher Hall", "text": "All test results verified and approved."},
16    {"author": "Operations Lead Grant", "text": "Emergency protocol draft shared via https://ops-share.scp"}
17 ]
18
19 reports_with_links = list(filter(
20     lambda r: 'http://' in r['text'] or 'https://' in r['text'],
21     reports
22 ))
23
24 import re
25
26 cleaned_reports = list(map(
27     lambda r: {
28         "author": r["author"],
29         "text": re.sub(r'https?:\/\/[^\s]+', '[ДАННЫЕ УДАЛЕНЫ]', r["text"])
30     },
31     reports_with_links
32 ))
33
34 print("Отчеты, содержавшие ссылки (после очистки):")
35 print("=" * 50)
36 for report in cleaned_reports:
37     print(f"Автор: {report['author']}")
38     print(f"Текст: {report['text']}")
39     print("-" * 50)

```

Рисунок 5 — листинг программы для задания 5

- На первом этапе использована функция `filter()` с лямбда-выражением, которое проверяет наличие подстрок `'http://'` или `'https://'` в тексте отчета
- На втором этапе использована функция `map()` с лямбда-выражением и регулярным выражением для замены всех URL на текст "[ДАННЫЕ УДАЛЕНЫ]"

1.6 Задание 6

Программа фильтрует список SCP-объектов, оставляя только те, которые требуют усиленных мер содержания (все, кроме класса "Safe").

```
1  scp_objects = [  
2      {"scp": "SCP-096", "class": "Euclid"},  
3      {"scp": "SCP-173", "class": "Euclid"},  
4      {"scp": "SCP-055", "class": "Keter"},  
5      {"scp": "SCP-999", "class": "Safe"},  
6      {"scp": "SCP-3001", "class": "Keter"}  
7  ]  
8  
9  # Фильтрация SCP-объектов, требующих усиленных мер содержания  
10 enhanced_containment_scps = list(filter(  
11     lambda scp: scp["class"] != "Safe",  
12     scp_objects  
13 ))  
14  
15 print("SCP-объекты, требующие усиленных мер содержания:")  
16 for scp in enhanced_containment_scps:  
17     print(f"{scp['scp']} – Класс: {scp['class']}")
```

Рисунок 6 – листинг программы для задания 6

- Использована функция filter() с лямбда-выражением в качестве условия фильтрации
- Лямбда-выражение проверяет, что значение ключа "class" не равно "Safe"

1.7 Задание 7

Программа сортирует инциденты по количеству задействованного персонала (по убыванию) и выбирает три наиболее ресурсоемких инцидента.

```

1 incidents = [
2     {"id": 101, "staff": 4},
3     {"id": 102, "staff": 12},
4     {"id": 103, "staff": 7},
5     {"id": 104, "staff": 20}
6 ]
7
8 sorted_incidents = sorted(
9     incidents,
10    key=lambda x: x["staff"],
11    reverse=True
12 )
13
14 top_3_incidents = sorted_incidents[:3]
15
16 print("Три наиболее ресурсоемких инцидента:")
17 for incident in top_3_incidents:
18     print(f"Инцидент ID: {incident['id']} – Персонал: {incident['staff']} чел.")

```

Рисунок 7 – листинг программы для задания 7

- Использована функция sorted() с параметрами: key=lambda x: x["staff"] - сортировка по ключу "staff", reverse=True - сортировка по убыванию
- Применен срез списка [:3] для выбора первых трех элементов (наиболее ресурсоемких)

1.8 Задание 8

Программа преобразует список кортежей (протокол, критичность) в список форматированных строк заданного вида.

```

1 protocols = [
2     ("Lockdown", 5),
3     ("Evacuation", 4),
4     ("Data Wipe", 3),
5     ("Routine Scan", 1)
6 ]
7
8 formatted_protocols = list(map(
9     lambda p: f"Protocol {p[0]} – Criticality {p[1]}",
10    protocols
11 ))
12
13 print("Протоколы безопасности:")
14 for protocol in formatted_protocols:
15     print(protocol)

```

Рисунок 8 – листинг программы для задания 8

- Использована функция `map()` с лямбда-выражением, которое форматирует каждый кортеж в строку вида "Protocol {name} - Criticality {level}"

1.9 Задание 9

Программа фильтрует список длительностей смен, оставляя только те, которые делятся от 8 до 12 часов включительно.

```
1 shifts = [6, 12, 8, 24, 10, 4]
2
3 # Фильтрация смен по длительности
4 valid_shifts = list(filter(
5     lambda duration: 8 <= duration <= 12,
6     shifts
7 ))
8
9 print(f"Все смены: {shifts}")
10 print(f"Смены длительностью от 8 до 12 часов: {valid_shifts}")
```

Рисунок 9 – листинг программы для задания 9

- Использована функция `filter()` с лямбда-выражением в качестве условия фильтрации
- Лямбда-выражение проверяет, что значение находится в диапазоне от 8 до 12 включительно

1.10 Задание 10

Программа находит сотрудника с наивысшим баллом психологической оценки, используя функцию `max()` с лямбда-выражением в качестве ключа сравнения.

```
1 evaluations = [
2     {"name": "Agent Cole", "score": 78},
3     {"name": "Dr. Weiss", "score": 92},
4     {"name": "Technician Moore", "score": 61},
5     {"name": "Researcher Lin", "score": 88}
6 ]
7
8 # Поиск сотрудника с наивысшей оценкой
9 top_performer = max(
10     evaluations,
11     key=lambda x: x["score"]
12 )
13
14 print(f"Сотрудник с наивысшей оценкой: {top_performer['name']}")
15 print(f"Балл: {top_performer['score']}")
```

Рисунок 10 – листинг программы для задания 10

- Использована функция `max()` с параметром `key=lambda x: x["score"]`, который указывает, что сравнение должно выполняться по ключу "score"
- Результат (словарь с данными сотрудника) сохраняется в переменную `top_performer`